

# Reviewer Pack — Minimal Order of Formal Language Automorphisms vs Monotone C...

Entry f13f502afce0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 13:42:30 UTC

## Minimal Order of Formal Language Automorphisms vs Monotone Circuit Lower Bounds for k-CLIQUE

Entry ID: f13f502afce0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 13:42:30 UTC

### 1. Conjecture

**Field A** (mathematical branch): Formal Language Theory (Automata Theory) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

#### Statement:

[“For every instance of k-CLIQUE, the minimal order of an automorphism group of a regular expression describing the complement of the clique’s induced subgraph is  $\Theta(k^{1/4})$ .”, ‘The minimal order of such an automorphism group is bounded by  $O(\log n)$  for any instance with at most  $n$  vertices.’, ‘For all instances with k-CLIQUE, the minimal order of an automorphism group is  $\Omega(n)$ .’]

#### Rationale (proposer’s reasoning):

[‘Formal language theory, particularly automata theory, has not been extensively applied to the study of monotone circuit complexity. This conjecture aims to bridge this gap and provide a potential new tool for understanding lower bounds.’, ‘Automorphism groups can capture symmetries in graphs, which might be related to the structure of k-CLIQUE graphs and their induced subgraphs. Exploring such a relationship could lead to novel insights into the complexity of monotone circuits.’, ‘The proposed mapping from graph instances to automorphism groups is computationally feasible, making it testable within the given constraints.’]

**Taxonomy category:** MONOTONE\_CLIQUE (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 1729f7a340dcde4d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The minimal order of an automorphism group for a regular expression describing the complement of a k-CLIQUE induced subgraph is either  $\Theta(k^{1/4})$  or bounded by  $O(\log n)$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** ADJACENT\_OK against 11 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - automorphism group regular expression complement clique monotone circuit complexity - k-clique automorphism group minimal order formal language theory monotone circuit lower bounds -  $\Theta(k^{\{1/4\}})$   $\Omega(n)$  automorphism group regular expression k-clique monotone circuits

**Top relevant hits considered:** - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/1402.5652v3>] Compact presentability of tree almost automorphism groups - [<http://arxiv.org/abs/1807.00547v3>] Realisation of groups as automorphism groups in categories - [<http://arxiv.org/abs/1405.2700v1>] Zero Excess and Minimal Length in Finite Coxeter Groups - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/2510.17487v1>] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [<http://arxiv.org/abs/1411.4413v2>] Observation of the rare  $B_s^0 \rightarrow \mu^+ \mu^-$  decay from the combined analysis of CMS and LHCb data

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math

def is_prime(n):
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True

def generate_primes(count):
    primes = []
    num = 2
```

```

while len(primes) < count:
    if is_prime(num):
        primes.append(num)
    num += 1
return primes

def random_k_clique(n, k):
    graph = [[0] * n for _ in range(n)]
    edges = set()
    nodes = list(range(n))
    while len(edges) < k:
        u, v = random.sample(nodes, 2)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            graph[u][v] = 1
            graph[v][u] = 1
            edges.add((u, v))
    return graph

def regular_expression_from_graph(n):
    # Placeholder for actual implementation
    return "regular_expression"

def automorphism_group_order(regular_expression):
    # Placeholder for actual implementation
    return [random.randint(1, n) for _ in range(random.randint(1, 5))]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    primes = generate_primes(30)
    results = []

    for n in [5, 10, 15, 20, 30, 40]:
        graph = random_k_clique(n, random.randint(1, n // 2))
        regular_expression = regular_expression_from_graph(n)
        orders = automorphism_group_order(regular_expression)

        for _, edges in graph:
            for order in orders:
                if not (1 <= order ** (1/4) <= n):
                    return {
                        "metric_name": "minimal_order",
                        "metric_value": None,
                        "instances_tested": 0,
                        "conjecture_holds": False,
                        "counterexample": "mapping_undefined"
                    }

            results.append(order)

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    conjecture_holds = all(1 <= order ** (1/4) <= n for _, edges in graph for order in automorphism_group_order(regular_expression))

    return {
        "metric_name": "minimal_order",
        "metric_value": mean,

```

```

        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or generate_primes(30)

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r is not None and (1 <= r ** (1/4) <= n)) / len(results)

    if all(r is not None for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(r is None or not (1 <= r ** (1/4) <= n) for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result is None or not (1 <= r ** (1/4) <= n))
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6705d78c.py", line 99, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6705d78c.py", line 67, in run_trial
    orders = automorphism_group_order(regular_expression)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6705d78c.py", line 57, in automorphism_group_order
    return [random.randint(1, n) for _ in range(random.randint(1, 5))]
            ^
NameError: name 'n' is not defined

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

## Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture’s validity. | next: Review and debug the test code to ensure it runs successfully and produces results.

## 11. Audit log (LLM calls)

**Total LLM calls:** 11

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13915        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13481        |
| 3  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 14037        |
| 4  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 9916         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8382         |
| 6  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 9435         |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12173        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10942        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11385        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10702        |
| 11 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 11343        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125710 ms total latency. Provider mix: {'ollama\_remote': 11}

*(full prompt+response transcripts available in research/audit/f13f502afce0.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f13f502afce0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/f13f502afce0.tar.gz (if generated)